UNIT-3

Dr. U. Padmavathi

Department of Computer Science & Engineering
School of Engineering

Shiv Nadar University Chennai

SHIV NADAR UNIVERSITY CHENNAI

SYNCHRONIZATION

- BASIC TERMS

- Cooperating process
- Concurrent execution
- Parallel execution
- Producer-Consumer Problem
- Buffer

- In case of bounded buffer, we need to keep track of buffer – counter variable.

- Counter is incremented – when adding an item to buffer.
- Counter is decremented – when removing an item from the buffer.

- Assume counter = 5
- Producer and consumer execute counter++ and counter- - concurrently.
- Counter = ?

- It can be 4,5, or 6.
- But the answer should be ----------.
- 5.

- How the instruction executes in machine language?
- Counter ++
- ```
  register1 = counter
  register1 = register1 + 1
  counter = register1
  ```

- Counter - -
- ```
  register2 = counter
  register2 = register2 - 1
  counter = register2
  ```

- S0: producer execute `register1 = counter` {register1 = 5}
  S1: producer execute `register1 = register1 + 1` {register1 = 6}
  S2: consumer execute `register2 = counter` {register2 = 5}
  S3: consumer execute `register2 = register2 - 1` {register2 = 4}
  S4: producer execute `counter = register1` {counter = 6 }
  S5: consumer execute `counter = register2` {counter = 4}

- Arrived at incorrect state. Counter == 4.
- Both processes manipulate the variable counter concurrently.
- Race condition.

- To guard against race condition – only one process should manipulate the variable at a time – synchronized.

CRITICAL-SECTION PROBLEM

- Critical section – segment of code that change common variables, update a table, write to a file and so on.
- When one process is in critical section, then no other process is allowed to execute in its critical section.
- No two processes are executing in its critical section at the same time.
- Design a protocol so that the processes can cooperate.

- The process must request permission to enter its critical section.
- The section of code implementing this request is the entry section.
- Critical section is followed by an exit section.
- Remaining code is the remainder section.

General Structure of a Process

```
do {
        entry section
            critical section
        exit section
            remainder section
} while (true);
```

- Solution to the critical section problem must satisfy three requirements:

- Mutual exclusion
- Progress
- Bounded waiting

PETERSON'S SOLUTION

- Software-based solution to critical section problem.
- This solution does not work correctly on modern architectures.
- It provides a good algorithmic description of solving the critical-section problem.
- This solution is restricted to two processes that alternate execution between their critical sections and remainder sections.

- Peterson’s solution requires the two processes to share two data items:

- int turn;
- boolean flag[2];

- turn – indicates whose turn it is to enter its critical section.
- flag – indicate if a process is ready to enter its critical section.

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn = = j);
        critical section
    flag[i] = false;
        remainder section
  } while (true);
```

- The value of turn determines which of the two processes is allowed to enter its critical section first.

SHIV NADAR UNIVERSITY CHENNAI

- Peterson’s solution is correct. Since,

- Mutual exclusion is preserved.
- Progress requirement is satisfied.
- Bounded-waiting requirement is met.

SYNCHRONIZATION HARDWARE

- Hardware based solution to critical section problem.
- Based on locking – protecting the critical regions through the use of locks.
- Test_and_set instruction.

```
boolean test_and_set (boolean *target)
        {
            boolean rv = *target;
            *target = TRUE;
            return rv:
        }
```

1. Executed atomically
2. Returns the original value of passed parameter
3. Set the new value of passed parameter to “TRUE”.

```
do {
        while (test_and_set(&lock))
            ; /* do nothing */
                /* critical section */
        lock = false;
                /* remainder section */
    } while (true);
```

- Satisfies mutual exclusion
- Does not satisfy bounded waiting

MUTEX LOCKS

- Hardware based solutions are complicated and inaccessible to application programmers.
- Operating system designers build software tools to solve the critical-section problem.
- Simplest tool – mutex locks.
- Mutex lock – protect critical regions and prevent race conditions.

- Process – acquire lock – release lock – acquire () – release ().
- Available – Boolean variable – indicate the lock is available or not.
- If the lock is available, a call to acquire () succeeds and the lock is then considered unavailable.
- A process that attempts to acquire an unavailable lock is blocked until the lock is released.

- do {

    acquire lock

    critical section

    release lock

    remainder section

  } while (true);

```
acquire() {
        while (!available)
            ; /* busy wait */
        available = false;
    }
```

```
release() {
        available = true;
    }
```

SHIV NADAR
UNIVERSITY
CHENNAI

- Disadvantage – busy waiting
- Busy waiting wastes CPU cycles.

- Also called “spinlock” – process spins while waiting for the lock to become available.

- Spinlock – advantage – no context switch is required.

SEMAPHORES

- Robust tool that provides process synchronization in a more sophisticated way.

- A semaphore S is an integer variable.

- Accessed through two atomic operations: wait () and signal ()
- Wait () → P
- signal () → V

```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

```
signal(S){
  S++;
}
```

- When one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

SHIV NADAR
UNIVERSITY
CHENNAI

Semaphore usage

- Two types of semaphores:
- Counting semaphore – range over an unrestricted domain.
- Binary semaphore – range only between 0 and 1.

- Counting semaphores can be used to control access to a given resource consisting of a finite number of instances.

- The semaphore is initialized to the number of resources available.

- Each process that wishes to use a resource performs a wait () operation on the semaphore.

- When the process releases the resource, it performs a signal () operation .

- When the count for semaphore goes to 0, all resources are being used.

- Semaphores are used to solve various synchronization problems.

- Example:
- P1 →s1
- P2 → s2

- We want to execute s2 after s1 – done using semaphores – initialize synch to zero.

```
P1:
S1;
signal(synch);
```

```
P2:
  wait(synch);
  S2;
```

Semaphore Implementation

- Wait () and signal () semaphores also suffer from busy waiting.
- To overcome busy waiting, the process can block itself.
- Block operation places a process into waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- Control is transferred to the CPU scheduler, which selects another process to execute.

- A process that is blocked – restarted when some other process executes a signal () operation.

- The process is restarted by a wakeup () operation, which changes the process from the waiting state to the ready state.

- The process is then placed in the ready queue.

- The semaphores are implemented as follows:

```
typedef struct{
   int value;
   struct process *list;
   } semaphore;
```

```
wait(semaphore *S) {
   S->value--;
   if (S->value < 0) {
      add this process to S->list;
      block();
   }
}
```

```
signal(semaphore *S) {
   S->value++;
   if (S->value <= 0) {
      remove a process P from S->list;
      wakeup(P);
   }
}
```

Deadlocks and Starvation

- The implementation of semaphores with waiting queues may result in a situation called deadlock.

- Two or more processes waiting indefinitely for an event that can be caused only by one of the waiting processes.

- Example:when P0 executes wait (Q), it must wait until P1 executes signal(Q).

P_0	P_1
wait(S);	**wait(Q);**
wait(Q);	**wait(S);**
...	**...**
signal (S);	**signal(Q);**
signal(Q);	**signal(S);**

SHIV NADAR UNIVERSITY CHENNAI

Starvation

- Another problem related to deadlocks is starvation or indefinite blocking.
- A situation in which processes wait indefinitely within the semaphore.
- Indefinite blocking occurs if we remove processes from the list associated with a semaphore in LIFO order.

Priority Inversion

- A scheduling challenge arises when a higher-priority process needs to read or modify kernel data that are currently being accessed by a lower-priority process.

- Situation becomes complex – if a lower-priority process is preempted in favor of another process with a higher priority.

- Example: L,M,H processes → L < M < H
- M – affect how long the process H must wait for L.
- Priority Inversion.

- Occurs only in systems with more than two priorities.

- Solution – have only two priorities – insufficient for general-purpose systems.

- Priority-inheritance protocol – allow process L to temporarily inherit the priority of process H, thereby preventing process M from preempting its execution.

Classic Problems of synchronization

- Bounded-Buffer Problem

- The producer and consumer processes share the following data structures:

- int n; buffers
- semaphore mutex = 1; access to buffers – mutual exclusion
- semaphore empty = n; number of empty buffers
- semaphore full =0. number of full buffers

Producer Process

```
do {
          ...
        /* produce an item in
next_produced */
          ...
        wait(empty);
        wait(mutex);
           ...
        /* add next produced to the
buffer */
           ...
        signal(mutex);
        signal(full);
     } while (true);
```

Consumer Process

```
Do {
        wait(full);
        wait(mutex);
           ...
        /* remove an item from buffer to
next_consumed */
           ...
        signal(mutex);
        signal(empty);
           ...
        /* consume the item in next
consumed */
           ...
     } while (true);
```

- The Readers-Writers Problem

- Readers – processes that want only to read the database.
- Writers – processes that want to update the database.

- Ensure that writers have exclusive access to the database while writing – synchronization problem is referred as readers-writers problem.

- The readers-writers problem has several variations.

- The simplest one – first - readers-writers problem – no reader be kept waiting unless a writer has already obtained permission to use the shared object.

- Second - readers-writers problem requires that once a writer is ready, that writer performs its write as soon as possible.

- The readers process share the following data structure:

- semaphore rw_mutex = 1;
- semaphore mutex = 1; [used to acquire a lock whenever the readcount is to be updated][no two process can modify the readcount at the same time]
- int read_count = 0;

```
do {
        wait(rw_mutex);
            ...
        /* writing is performed
*/
            ...
        signal(rw_mutex);
    } while (true);
```

```
do {
        wait(mutex);
        read_count++;
        if (read_count == 1)
          wait(rw_mutex);
        signal(mutex);
           ...
        /* reading is performed */
           ...
        wait(mutex);
        read count--;
        if (read_count == 0)
        signal(rw_mutex);
        signal(mutex);
      } while (true);
```

SHIV NADAR UNIVERSITY CHENNAI

- Reader-writer locks are useful in the following situation:

- In applications where it is easy to identify which processes only read shared data and which processes only write shared data.

- In applications that have more readers than writers.

- Dining-Philosophers Problem:

- Philosophers spend their time thinking and eating.
- When a philosopher thinks, she does not interact with her colleagues.
- When a philosopher is hungry, she tries to pick up the two chopsticks closest to her.
- A philosopher can pick up only one chopstick at a time.
- She cannot pickup a chopstick that is already in the hands of her neighbor.

- Solution - Represent each chopstick with a semaphore.
- A philosopher tries to grab a chopstick by executing a wait () operation on that semaphore.
- She releases her chopsticks by executing the signal () operation on the appropriate semaphores.
- Semaphore chopstick[5];

```
do {
    wait (chopstick[i] );
    wait (chopStick[ (i + 1) % 5] );

            //  eat

    signal (chopstick[i] );
    signal (chopstick[ (i + 1) % 5] );

            //  think

} while (TRUE);
```

- This solution guarantees that no two neighbors are eating simultaneously.

- Still it results in deadlock.

- Suppose if all five philosophers become hungry at the same time, and each grabs her left chopstick.

- Several possible remedies to the deadlock problem are:
- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
- Use an asymmetric solution – odd → first left then right – even → first right then left.

Monitors

- Semaphores – provide a convenient and effective mechanism for process synchronization.

- But using semaphores incorrectly can result in errors that are difficult to detect.

- Example: 1. Process interchanges the order.
- 2. Process replaces signal(mutex) with wait (mutex).
- 3. The process omits signal (mutex) or wait (mutex).

- To deal with such errors, researchers have developed high-level language constructs – Monitors.

- Monitors – ADT – includes a set of programmer-defined operations that are provided with mutual exclusion within the monitor.

- Monitor also declares variables whose values define the state of an instance of that type, along with the bodies of functions that operate on those variables.

Syntax of a monitor

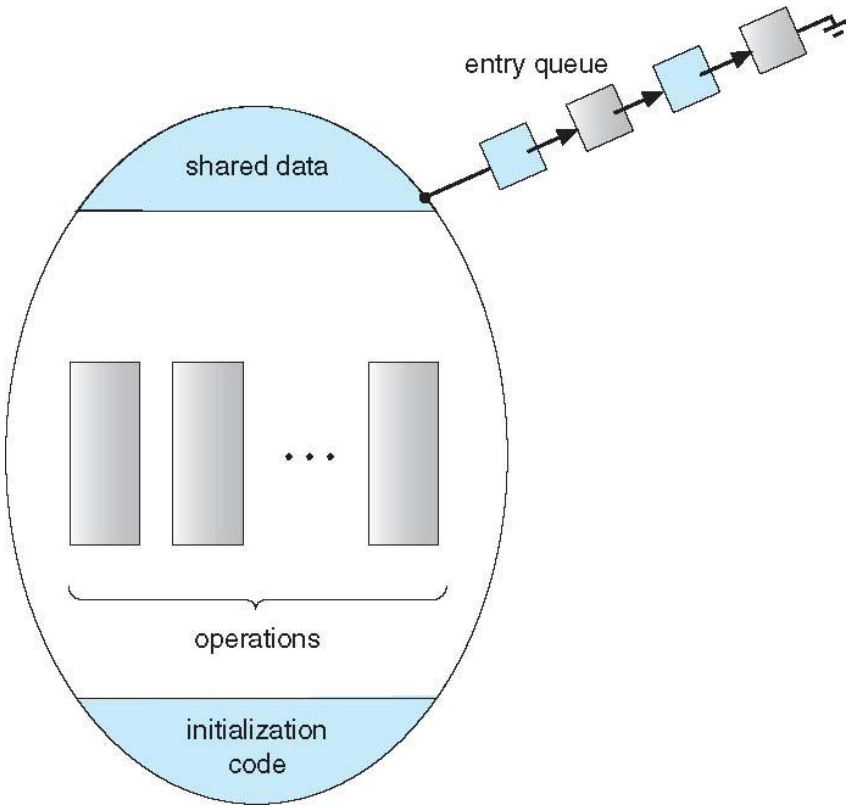
```
monitor monitor-name
{
        // shared variable declarations
        function P1 (…) { …. }

        function Pn (…) {……}

    Initialization code (…) { … }
        }
}
```

SHIV NADAR UNIVERSITY CHENNAI

- A function defined within a monitor can access only those variables declared locally within the monitor and its formal parameters.
- The local variables of a monitor can be accessed by only the local functions.
- Monitor construct ensures that only one process at a time is active within the monitor.
- Monitors are provided with condition construct – to provide additional synchronization mechanisms.

- A programmer who needs to write a tailor-made synchronization scheme can define one or more variables of type condition:

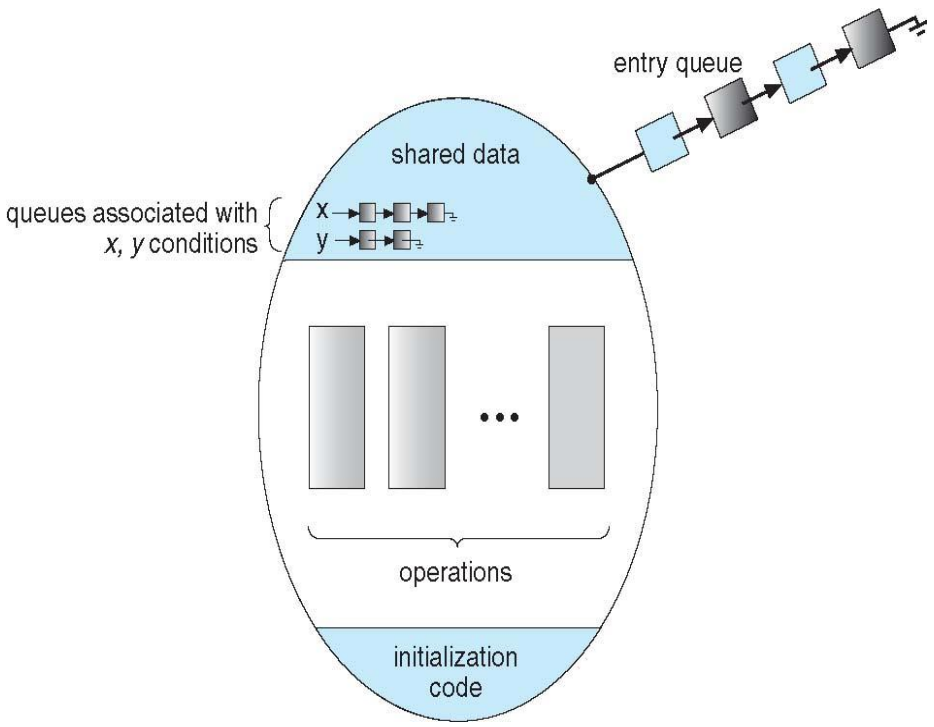
- condition x,y;



- The only operation that can be invoked on a condition variable are wait () and signal ().

```
x.wait ();
x.signal ();
```

- x.wait () means that the process invoking this operation is suspended until another process invokes x.signal ()



Dining-Philosophers solution using Monitors

- Imposes the restriction that a philosopher may pick up her chopsticks only if both of them are available.
- To code this, we need three states: eating, hungry, thinking
- Data structure: enum { THINKING, HUNGRY, EATING } state [5];
- Condition self[5]; → allows philosopher i to delay herself when she is hungry but is unable to obtain the chopstick she needs.

```
monitor DiningPhilosophers
{
        enum { THINKING; HUNGRY, EATING)
state [5] ;
        condition self [5];


void pickup (int i) {
                state[i] = HUNGRY;
                test(i);
                if (state[i] != EATING) self[i].wait;
}



 void putdown (int i) {
                state[i] = THINKING;
            // test left and right neighbors
                test((i + 4) % 5);
                test((i + 1) % 5);
}
```

```
void test (int i) {
if ((state[(i + 4) % 5] != EATING) &&
                (state[i] == HUNGRY) &&
                (state[(i + 1) % 5] != EATING) )
                {
                    state[i] = EATING ;
                        self[i].signal () ;
                }
  }




 initialization_code() {
                for (int i = 0; i < 5; i++)
                state[i] = THINKING;
               }
}
```

SHIV NADAR UNIVERSITY CHENNAI

- Each philosopher i must invoke the operations pickup () and putdown () in the following sequence:

```
DiningPhilosophers.pickup(i);
.
.
eat
.
.
DiningPhilosophers.putdown(i);
```

DEADLOCKS

- Deadlock – waiting processes are not allotted resources for long time.

- System Model
- A system consists of a finite number of resources to be distributed among a number of competing processes.

- The resources may be partitioned into several types, each consisting of some number of identical instances.
- Example: if a system has two CPUs then the resource type CPU has two instances.

- Process request an instance – any instance of that type should satisfy the request.
- A process must request the resource before using it and must release the resource after using it.
- A process may request as many resources as it requires to carry out its designated task.
- The number of resources requested may not exceed the total number of resources available in the system.

- A process may utilize a resource in only the following sequence:

- Request
- Use
- Release

- The request and release of resources may be system calls.

- A set of processes is in a deadlocked state when every process in the set is waiting for an event that can be caused only by another process in the set.

DEADLOCK CHARACTERIZATION

- In deadlock, processes never finish executing, and system resources are tied up, preventing other jobs from starting.

1. Necessary Conditions
2. Resource-Allocation Graph

Necessary Conditions

- Mutual Exclusion – only one process at a time can use the resource.
- Hold and wait – holding one resource and waiting for other resource being held by other processes.
- No preemption – Resources cannot be preempted.
- Circular wait – A set of processes wait in a circular fashion.

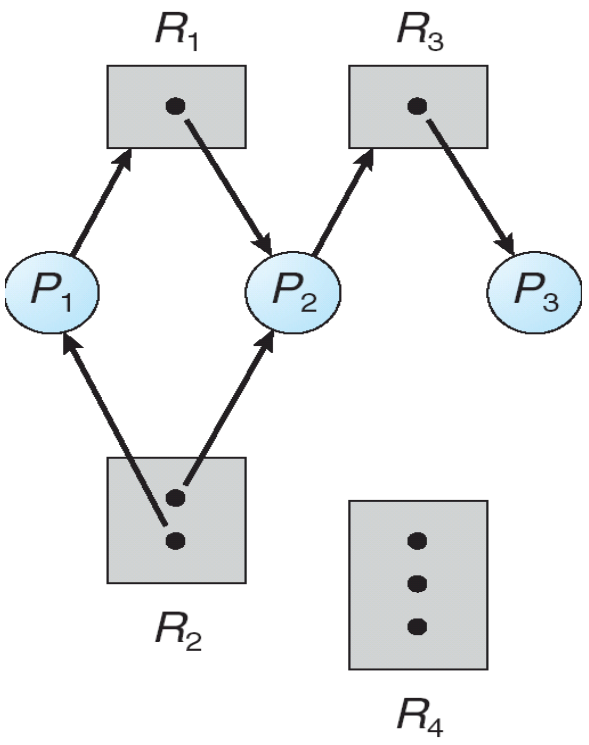
Resource Allocation Graph

- Deadlocks can be described using a directed graph called system resource-allocation graph.

- Vertices → Two types of nodes: P = {P1, P2,…..,Pn}, R = {R1, R2,….,Rm}.

- An edge from Process Pi to Resource type Rj is denoted by Pi → Rj – Request edge
- A directed edge from resource type Rj to Process Pi is denoted by Rj → Pi – Assignment edge

- Each process Pi – circle
- Each resource type Rj – rectangle.
- Each instance – dot



- Request edge – point to rectangle
- Assignment edge – designate dot

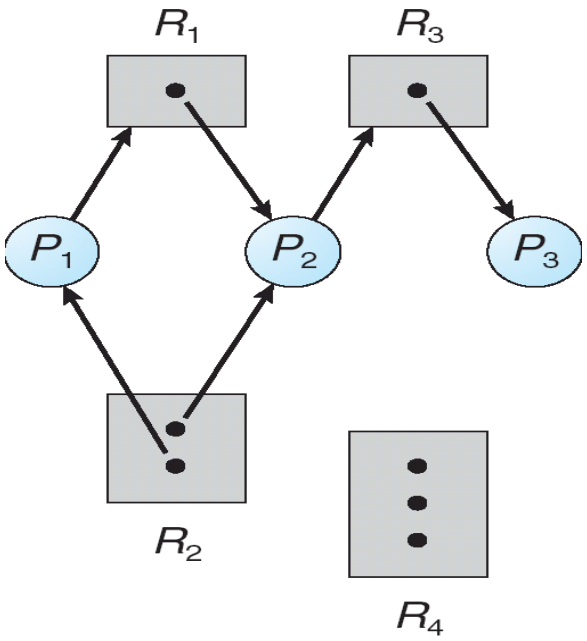
- Request – request edge – fullfill – transferred to assignment edge.

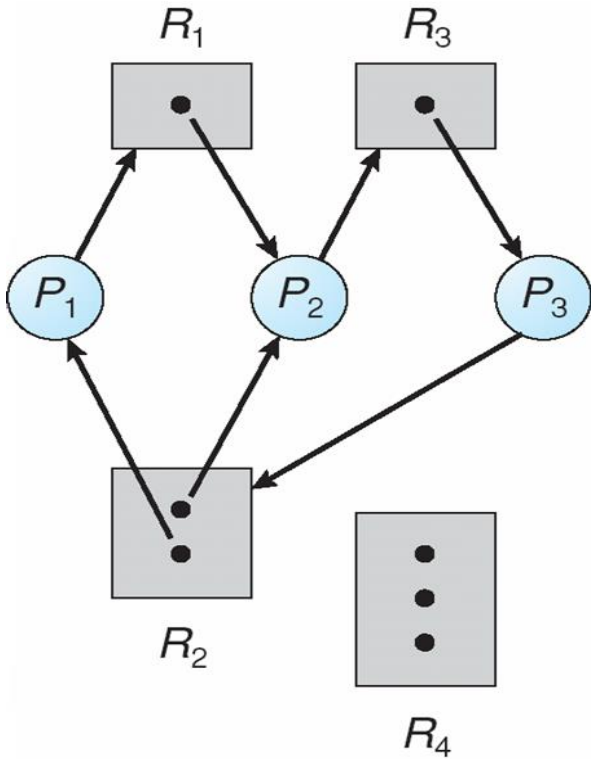
- Graph – cycle – deadlock may exist
- No cycle – no deadlock

- If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred – necessary and sufficient condition for deadlock.

- If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred – cycle is a necessary but not a sufficient condition for deadlock.

- P3 requests R2



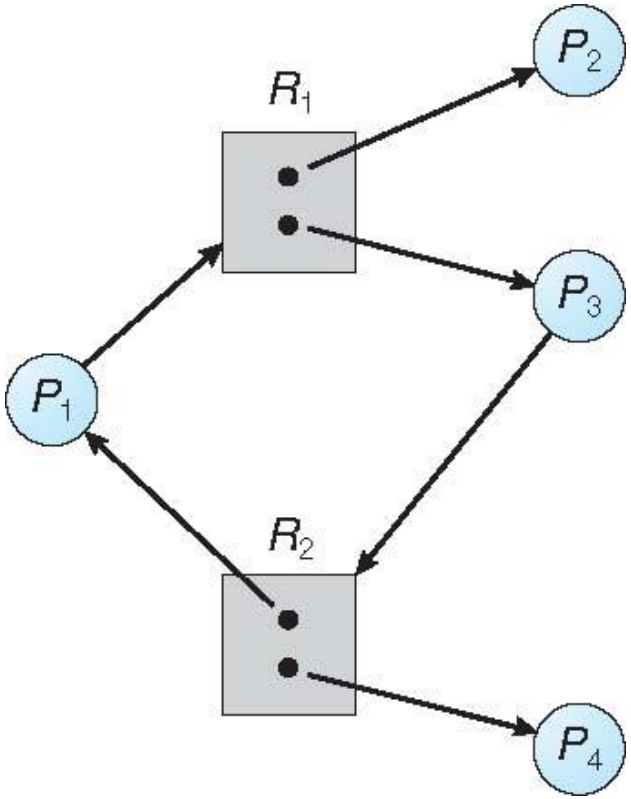


- Two minimal cycles exist in the system:
- P1 → R1 → P2 → R3 → P3 → R2 → P1
- P2 → R3 → P3 → R2 → P2

- Cycle exists
- P1 → R1 → P3 → R2 →P1

- No deadlock

- No cycle – No deadlock
- Cycle – may or may not be deadlock



Methods for Handling Deadlocks

- We can deal with deadlock problem in one of the three ways:
- We can use a protocol to prevent or avoid deadlocks, ensuring that the system will never enter a deadlocked state.
- We can allow the system to enter a deadlocked state, detect it and recover.
- We can ignore the problem altogether and pretend that deadlocks never occur in the system.

Deadlock Prevention

- Mutual Exclusion

- Mutual Exclusion condition must hold.
- Atleast one resource must be nonsharable.
- We cannot prevent deadlocks by denying the mutual-exclusion condition, because some resources are intrinsically nonsharable.

- Hold and Wait

- Ensure - Hold and wait never occurs – guarantee – whenever a process requests a resource, it does not hold any other resources.

- Protocol 1 - Requires each process to request and be allocated all its resources before it begins execution.

- Protocol - 2Allow a process to request resources only when it has none.

- Example: writing to a file from disk and printing it.

SHIV NADAR UNIVERSITY CHENNAI

- Both protocols have two main disadvantages:

- Resource-utilization may be low.
- Starvation is possible

- No Preemption

- To ensure this condition does not hold:
- If a process is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources the process is currently holding are preempted.

- If a process requests some resources – check available – not – check whether allotted to some other process that is waiting for additional resources – preempt desired resources – otherwise the requesting process must wait.

- Circular wait

- One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.

- R= {R1, R2, ….Rm}
- F:R → N

- Example: F(tapedrive) = 1
- F(disk drive) = 5
- F(Printer) = 12

Deadlock Avoidance

- Deadlock prevention algorithms prevent deadlocks by limiting how requests can be made.
- The limits ensure that atleast one of the necessary conditions for deadlock cannot occur.
- Side effects → low device utilization and reduced system throughput.
- Deadlock avoidance – require additional information about how resources are to be requested.

- Example: system – one tape drive and one printer
- Process P → first tapedrive then printer
- Process Q → first printer then tape drive

- The system can decide for each request whether or not the process should wait in order to avoid a possible future deadlock.

- Each request requires that in making this decision the system consider the resources currently available, the resources currently allocated to each process, and the future requests and releases of each process.

Deadlock Avoidance algorithms

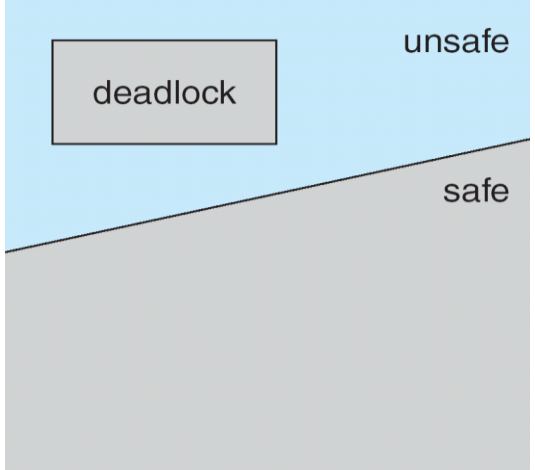
- Safe State

- A state is safe if the system can allocate resources to each process in some order and still avoid a deadlock.

- A system is in a safe state if there exists a safe sequence.

- Process – {P1, P2, ….Pn}

- A safe state is not a deadlocked state.
- A deadlocked state is an unsafe state.
- Not all unsafe states are deadlock states.
- An unsafe state may lead to a deadlock.



- Example:

-

	Maximum Needs	Current Needs
P0	10	5
P1	4	2
P2	9	2

- <P1, P0, P2> - safe state

Resource-Allocation-Graph Algorithm

- In addition to the request and assignment edges, there is another edge called claim edge.
- A claim method Pi → Rj indicates that process Pi may request resource Rj at some time in the future.
- When process Pi requests resource Rj, the claim edge Pi → Rj is converted to a request edge.
- When a resource Rj is released by Pi, the assignment edge Rj → Pi is reconverted to a claim edge Pi → Rj.

- Before Process Pi starts executing, all its claim edges must already appear in the resource-allocation graph.

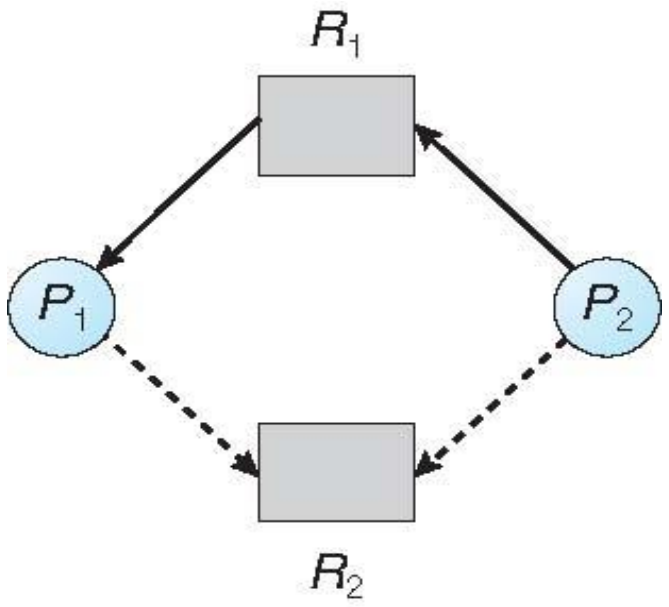
- Process Pi requests resource Rj. The request can be granted only if converting the request edge Pi → Rj to an assignment edge Rj → Pi does not result in the formation of a cycle in the resource-allocation graph.

- Cycle detection algorithm is used to check for safety.

- If there is no cycle → allocation → safe state.

- If there is cycle → allocation → unsafe state.

- If P2 requests R2, then if it is allotted, it may Result in a cycle → unsafe state.



Banker’s Algorithm

- Resource-allocation-graph is not applicable to a resource-allocation system with multiple instances of each resource type.

- Banker’s algorithm

- Several data structures must be maintained for implementing banker's algorithm:

- Available → vector of length m → indicates the number of available resources.

- Max → nxm matrix → maximum demand of each process

- Allocation → nxm matrix → resources currently allocated

- Need → nxm matrix → remaining resource need

- Need[i][j] = Max[i][j] – Allocation[i][j]

Safety algorithm

1. Let Work and Finish be vectors of length m and n, respectively. Initialize:
   Work = Available
   Finish [i] = false for i = 0, 1, …, n- 1

2. Find an i such that both:
   (a) Finish [i] = false
   (b) $Need_i \leq Work$
   If no such i exists, go to step 4

3. Work = Work + $Allocation_i$
   Finish[i] = true
   go to step 2

4. If Finish [i] == true for all i, then the system is in a safe state

Resource-Request Algorithm

Request$_i$ **= request vector for process** P_i**. If** ***Request***$_i$ **[*j*] = *k* then process** P_i **wants *k* instances of resource type** R_j

1. **If** ***Request***$_i \leq$ ***Need***$_i$ **go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim**

2. **If** ***Request***$_i \leq$ ***Available*****, go to step 3. Otherwise** P_i **must wait, since resources are not available**

3. **Pretend to allocate requested resources to** P_i **by modifying the state as follows:**

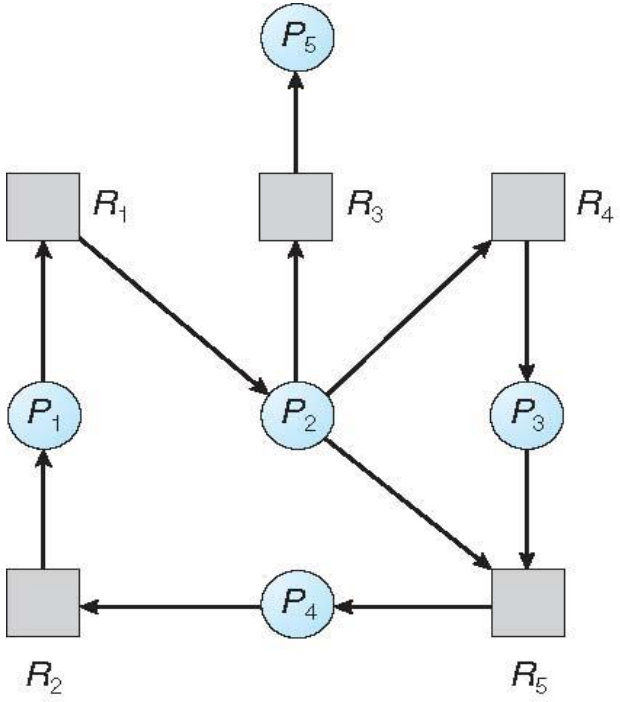
$$Available = Available - Request_i;$$
$$Allocation_i = Allocation_i + Request_i;$$
$$Need_i = Need_i - Request_i;$$

- **If safe $\Rightarrow$ the resources are allocated to** P_i
- **If unsafe $\Rightarrow$** P_i **must wait, and the old resource-allocation state is restored**

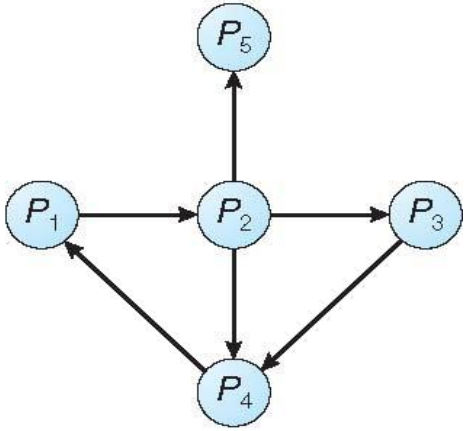
Deadlock Detection

- Single instance of Each Resource Type

- Variant of resource allocation graph – wait-for graph



(a)



(b)

- Several instances of a Resource type

- Wait-for graph is not suitable. So employ deadlock detection algorithm.

- Available
- Allocation
- Request

Deadlock Detection Algorithm

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively Initialize:

(a) *Work* = *Available*

(b) For $i = 1,2, \ldots, n$, if $Allocation_i \neq 0$, then
Finish*[i] = *false*; otherwise, *Finish*[i] = *true

2.Find an index *i* such that both:

(a) *Finish*[*i*] == *false*

(b) $Request_i \leq Work$

If no such *i* exists, go to step 4

3. Work = Work + $Allocation_i$
Finish*[*i*] = *true
go to step 2

4.If *Finish[i] == false*, for some *i*, $1 \leq i \leq n$, then the system is in deadlock state. Moreover, if *Finish*[*i*] == *false*, then P_i is deadlocked

SHIV NADAR UNIVERSITY CHENNAI

Recovery from Deadlock

- When a deadlock is detected:
- Inform the operator that deadlock has occurred and let the operator deal with the deadlock manually.
- Let the system recover form the deadlock automatically.
- Simply abort one or more processes to break the circular wait.
- Preempt some resources from one or more of the deadlocked processes.

Process Termination

- Abort all deadlocked processes: break deadlock – great expense.

- Abort one process at a time until the deadlock cycle is eliminated: considerable overhead – after each process is aborted, run deadlock-detection algorithm.

- Many factors may affect which process is chosen:

- What the priority of the process is
- How long the process has computed and how much longer the process will compute before completing its designated task.
- How many and what types of resources the process has used
- How many more resources the process needs in order to complete.
- How many processes will need to be terminated
- Whether the process is interactive or batch

Resource Preemption

- To eliminate deadlocks using resource preemption, we successively preempt resources from processes and give these resources to other processes until the deadlock cycle is broken.

- If preemption is required to deal with deadlocks, then three issues need to be addressed.
- Selecting a victim
- Rollback
- Starvation

THANK YOU